Лабораторная работа №1. Спецификация и архитектурное проектирование многоагентной системы ResearchRadar MAS

Дисциплина «Теория многоагентных систем»

Отчёт по фактически реализованному первому этапу. Реквизиты студента, группы и преподавателя не указаны в предоставленных материалах.

30 октября 2026 года

1. Цель и прикладная задача

Цель работы — выбрать задачу, формально описать MAC, построить диаграмму ролей и граф коммуникаций, обосновать архитектуру с учётом последующих лабораторных работ. Основной источник требований — методические рекомендации преподавателя (project_guide/Лабораторные-MAC.pdf, с. 1–2).

ResearchRadar MAS помогает исследователю начать обзор темы: получить научные записи и связанные по поисковому запросу новости, видеть происхождение и полноту данных. На первом этапе система не устанавливает доказательных связей между ними и не оценивает научную достоверность утверждений.

2. Требования и границы реализации

Вход — непустой вопрос или тема. Выход — структурированный JSON с двумя типами материалов, ошибками источников, состояниями агентов и трассой сообщений. DEMO воспроизводим без сети. LIVE обращается к OpenAlex и GDELT. Источники возвращают преимущественно метаданные; содержание публикации не выводится из одного заголовка. Ключ OpenRouter хранится вне репозитория.

Архитектура должна позволять позже сравнить не менее двух способов распределения задач, исследовать конфликт за ресурс и провести диверсионные испытания. Эти эксперименты в лабораторной №1 не выполнялись.

3. Формальная модель MAC

Обозначим MAC = (A, E, M, R, T, δ). A — множество агентов; E — внешние источники и файловое хранилище результата; M — структурированные сообщения; R — разрешённые направленные маршруты; T — задачи исследования; δ — переходы локального состояния. Глобальное состояние включает запрос, очереди, задачи, материалы и трассу. У каждого агента есть собственное состояние (phase, handled, last_error). Допустимые переходы: idle → busy → done либо idle → busy → failed. Программная проверка отклоняет остальные переходы.

Роль	Цель и знания	Действия и полномочия
Координатор	Декомпозировать запрос, собрать ответы; знает запрос и сообщения	Назначает две задачи, принимает результаты, вызывает синтезатор
Научный агент	Найти научные записи; знает свой запрос и ответ OpenAlex	Ищет и нормализует публикации, отвечает координатору
Новостной агент	Найти новости; знает свой запрос и ответ GDELT	Ищет и нормализует новости, отвечает координатору
Синтезатор	Выпустить ограниченную сводку; знает два набора материалов	Считает записи, обозначает пределы доказательств

Роль — абстрактная функция в MAC, экземпляр — конкретный объект Python. В текущей версии каждой роли соответствует один экземпляр. Поисковые агенты работают конкурентно; координатор выбирает исполнителей по компетенции. Это базовая политика для будущего сравнения, а не результат эксперимента ЛР №2.

4. Коммуникации и жизненный цикл

Сообщение содержит id, sender, recipient, kind, request_id, payload и correlation_id. Маршрутизатор проверяет разрешённость направления, пишет событие в трассу и доставляет сообщение в личную очередь. Ответ содержит корреляцию с задачей. Поисковые агенты не могут напрямую назначать задачи друг другу.

Последовательность: координатор создаёт научную и новостную задачи; оба агента получают сообщения и независимо запрашивают свои источники; координатор собирает результаты или ошибки, удаляет дубликаты по исходному идентификатору; синтезатор выпускает сводку; CLI сохраняет JSON. Ошибка внешнего источника переводит соответствующего агента в failed, но другой источник и итоговый файл остаются доступны.

5. Диаграмма ролей

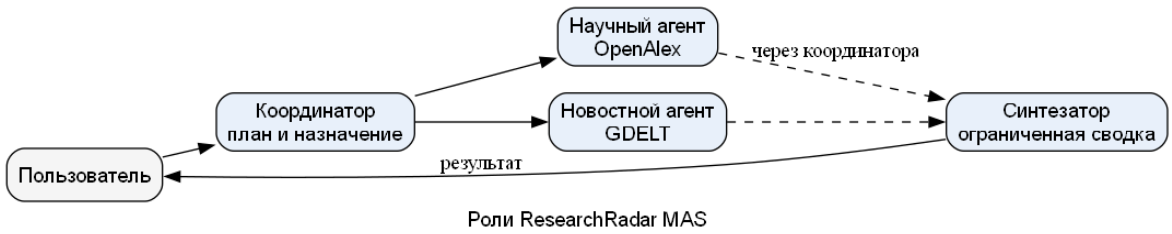


Рисунок 1 — Роли и общий поток результатов. Пунктирные связи проходят через координатора.

6. Направленный граф коммуникаций

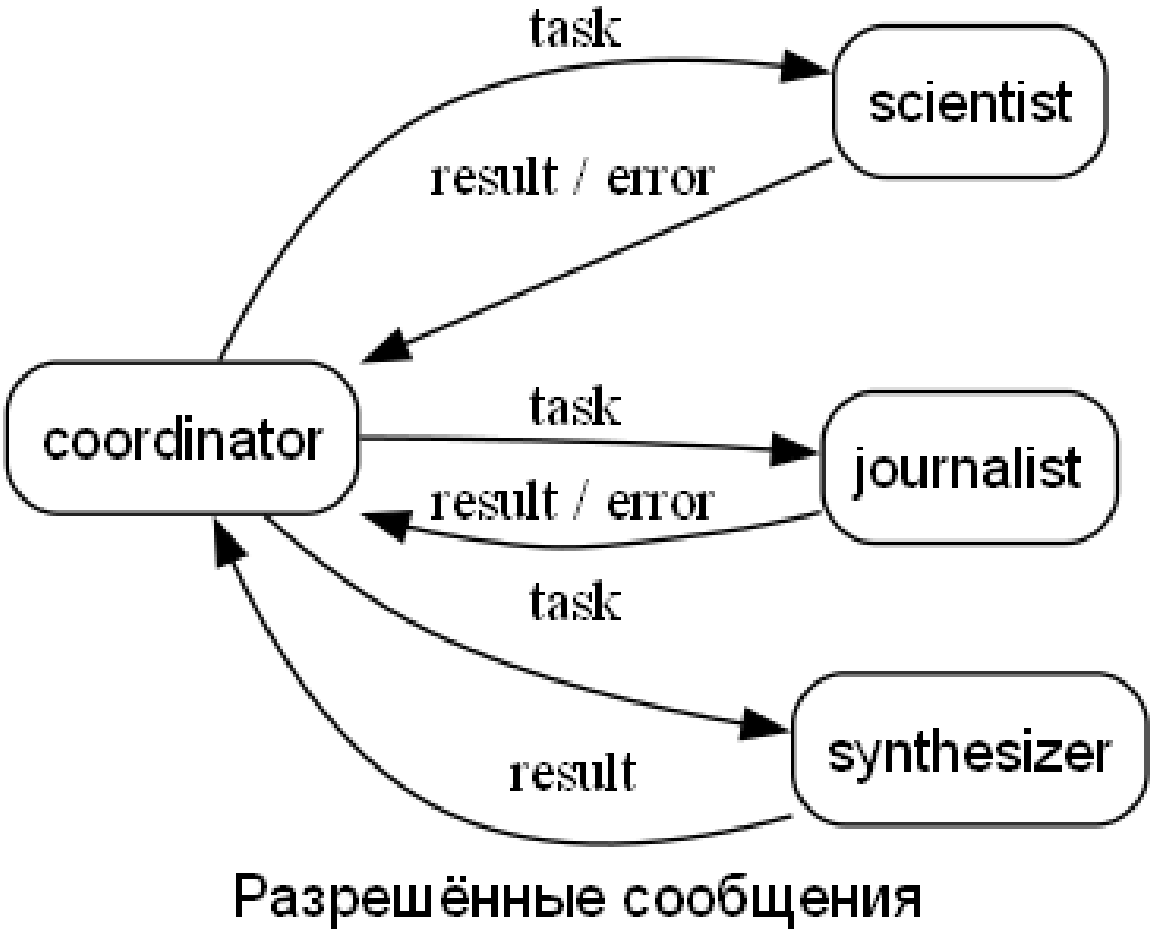


Рисунок 2 — Разрешённые направления структурированных сообщений.

7. Обоснование архитектуры

Выбрана гибридная MAC на Python с явным роутером и адаптерами источников. Python выполняет детерминированную маршрутизацию, контроль состояний, нормализацию и журналирование. Общий клиент Qwen через OpenRouter подготовлен для интеллектуальных действий, но не встроен в обязательный сквозной сценарий: без него демонстрация воспроизводима и не тратит средства. Специализированный агентный фреймворк для этой лабораторной не нужен, поскольку он скрыл бы исследуемые правила взаимодействия.

Задачи, сообщения, роли, источники и состояние оформлены отдельно. Это позволяет заменить политику назначения в ЛР №2, ввести заявки на ограниченные ресурсы в ЛР №3 и внедрять отказовые сценарии с анализом трассы в ЛР №4. Централизованный координатор остаётся единой точкой отказа, а трасса пока не хранит полные снимки состояния.

8. Реализация и проверка

Реализованы четыре независимые агентные сущности с личными очередями и состояниями, белый список маршрутов, два адаптера источников, нормализация научных записей и новостей, DEMO fixtures, JSON-результат и ограниченный OpenRouter-клиент. Синтезатор не формулирует исследовательских выводов по метаданным.

Проверено на Windows с Python 3.14: 6 модульных тестов прошли; offline demo создал по одной синтетической записи двух типов и трассу из 6 сообщений. LIVE smoke-тест: OpenAlex вернул 3 записи; GDELT вернул HTTP 429 (ограничение частоты), что отражено в результате. Следовательно, работа новостного LIVE-источника пока не подтверждена. Платных вызовов OpenRouter не выполнялось; фактические расходы — 0 USD.

Команды: py -3.14 -m pip install -e .; py -3.14 -m unittest discover -s tests -v; py -3.14 -m researchradar.cli 'multi agent systems' --mode demo; для сетевой проверки заменить demo на live.

9. Выводы и материалы

Требования ЛР №1 к постановке задачи, формальной спецификации, ролям, диаграмме, графу и обоснованию архитектуры выполнены. Реализованный срез позволяет показать обмен сообщениями и различие полномочий агентов. Ограничения: нет полноценного интеллектуального анализа, сравнения механизмов координации, переговоров и диверсионных испытаний; GDELT LIVE требует дополнительной диагностики.

Исходный код, исходники схем и инструкции: https://github.com/KAndyD/ResearchRadar_MAS. Архитектурная спецификация: docs/architecture.md. Демонстрационные материалы: run_output/result.json (генерируется командой DEMO). Ссылка на GitHub Release будет добавлена после подтверждённой публикации.